

ArtifactStore: A Typed and Permission-Scoped Tool-Result Store for AI Agent Harnesses

System architecture report — DBMS course prototype

May 2026

1 Abstract

Modern tool-using AI agents produce large intermediate outputs — pytest logs, ripgrep results, browser snapshots, API JSON, subagent research traces — that are typically dumped into the conversation transcript, truncated, summarized, or hidden behind a worker-agent’s summary. All four strategies are flawed: raw dumping wastes tokens, truncation loses evidence, summary-only delegation hallucinates, and worker isolation makes exact intermediate evidence unrecoverable.

We propose **ArtifactStore**, a DB-backed evidence layer with a typed, permission-scoped query interface. Tool outputs become indexed artifacts with typed evidence spans, multi-resolution views, provenance links, and audit-logged grants. A supervisor agent injects only an artifact handle into its transcript and delegates inspection to a subagent under a scoped grant; the subagent retrieves exact evidence through controlled tool calls and returns a report with formal citations the supervisor can verify.

Novelty positioning: agent-memory systems like **MemGPT** / **Letta** and **LangGraph checkpointers** persist intermediate state across turns but expose it as freeform text with no typed span, no citation primitive, and no per-read access surface. Capability-based access control (Dennis & Van Horn, 1966; **seL4**, **Capsicum**) and row-level security (Postgres RLS, Oracle VPD) are mature in OS and DBMS contexts but are not applied to agent tool outputs. The contribution is not any one of these layers but the recombination — typed evidence spans, capability-scoped reads, span-level citations, and append-only audit logs in a single substrate sized for an agent-harness use case, where the runtime (not the LLM) is the policy enforcer.

Scope of the empirical claims: live evaluation is 132 runs against one provider class (DeepSeek V4 Pro through the Anthropic-compatible Messages endpoint) on five captured fixtures spanning 407–9,609 raw tokens, 3 reps per (fixture × baseline) cell at **temperature=1.0**. The 9.6K fixture has n=3 per cell with wide Wilson 95% CIs (e.g., **[0.21, 0.94]** for 2/3); the structural claims (only B4/D3 produce verifiable citations and audit-log signal) hold by construction, not by reps. We have not run a second model class or a larger long-context fixture (≥30K tokens); both are flagged as limitations rather than future work.

Headline results within those bounds: §11.1 single-agent (5 fixtures × 5 baselines × 3 reps = 75 runs) shows B4 (ArtifactStore) reaches 12/15 (80%, Wilson 95% CI [0.55, 0.93]) task success and 0.82 avg evidence recall across the suite; B1 (raw injection) ties at 15/15 (100%, CI [0.80, 1.00]) — **no statistically defensible claim that B1 or B4 has an aggregate edge at n=15**. The robust empirical claim is that the summary baselines (B2/B3/B3′) collapse to ≤47% suite-wide success and to 0–33% on the two fixtures ≥3.5 K raw tokens, while B1 and B4 hold. §11.2 supervisor↔subagent (5 fixtures × 4 strategies × 3 reps = 60 runs) shows D3 (scoped ArtifactStore delegation) holds parent context bounded at 6 K tokens even on the 9.6 K fixture (D2 balloons to 25 K in fresh runs — confirmed within ±3% of the prior draft’s number). §11.3 adversarial stress (10 offline scenarios) shows

zero unauthorized reads succeed. A real LLM-summary baseline B3'/D1' was added at reviewer request and does **not** beat deterministic B3/D1 on evidence-recall at this prototype's summarizer budget — but the **structural** advantages of B4/D3 (citation verifiability, audit-log signal, span-aware selective redaction) survive any summary quality and are the load-bearing claim of this report.

Implementation: 2.0 kLOC of Python on SQLite + FTS5, plus a 150 LOC client-side tool-use loop. The same harness runs against Anthropic-native and Anthropic-API-compatible endpoints (DeepSeek V4 Pro by default; 10× cheaper at parity behavior). 171 tests pass — unit, integration, offline e2e, CLI integration (subprocess and in-process), adversarial stress, eval-baseline unit tests, FTS5-fallback robustness, and review-driven regression tests for self-review-discovered weaknesses (long-line truncation, sensitivity self-labeling bypass, fence-post budget enforcement, citation case-handling). A target-leak control sweep confirms the eval's headline numbers are robust to whether the agent is told which failure to focus on.

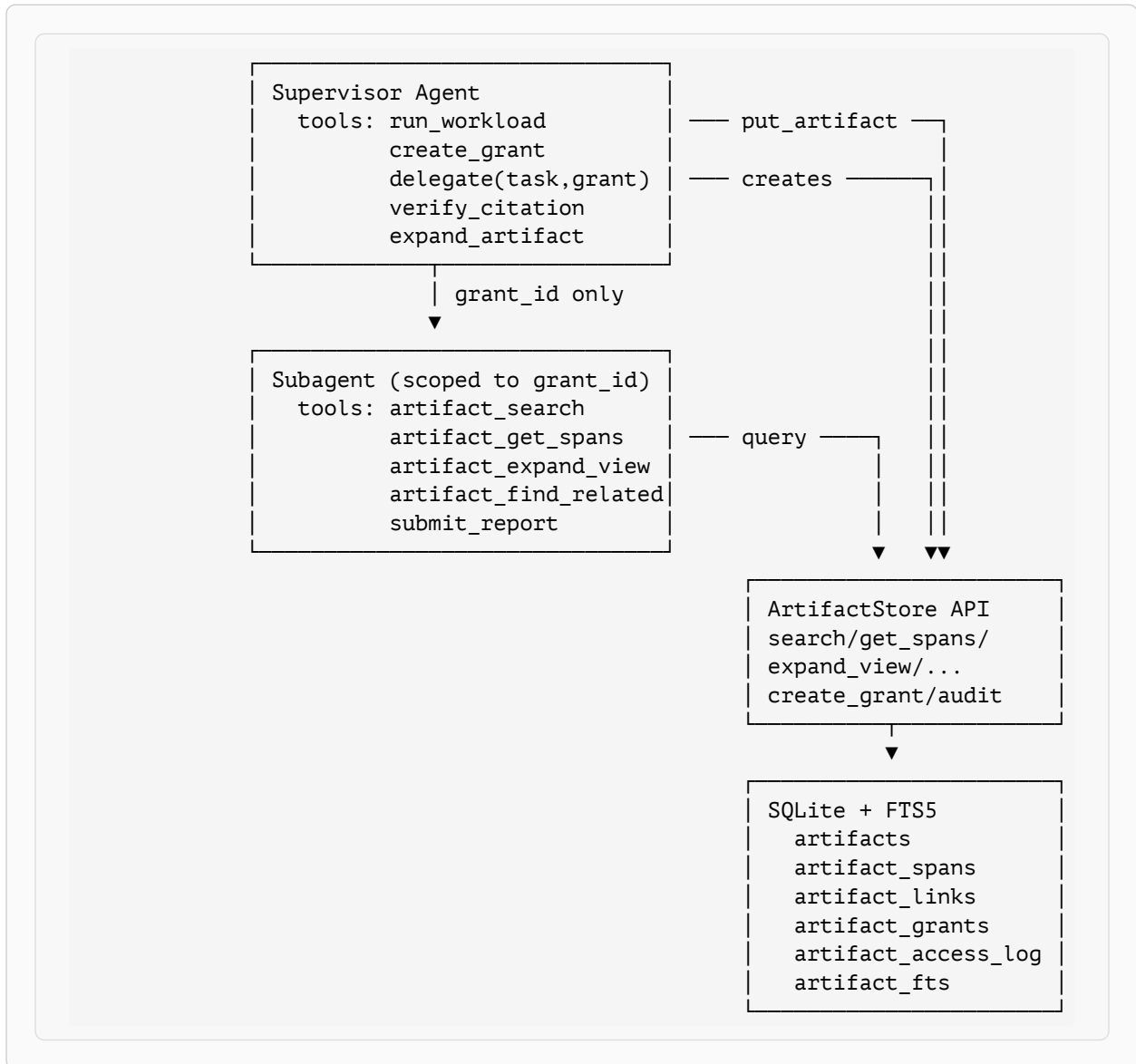
2 Motivation and Thesis

Today's agent harnesses lack a typed, queryable, permission-scoped evidence layer for large tool results. The four canonical handling strategies all fail in distinct ways:

Strategy	Failure mode	Cost shape
Raw dump (B1)	Doesn't scale; pollutes context	Linear in fixture size
Truncation (B2)	Loses evidence past the cap	Bounded but evidence-blind
LLM summary (B3)	Hallucinates or omits the decisive line	+1 LLM call per artifact, plus quality variance
Subagent isolation	Exact evidence unrecoverable in parent	Lower parent tokens; opaque audit

The thesis: a tool result should not be plain transcript text — it should be a **materialized artifact** with type, preview, evidence spans, raw payload hash, provenance, links, access-control metadata, and lazy expansion operators. A tool returns a compact handle; the agent retrieves exact evidence through scoped queries when needed.

3 System Overview



Listing 1: Two-agent topology with the ArtifactStore boundary. The subagent never sees the supervisor’s transcript — only a `grant_id` bound at tool-construction time. Every read flows through `ArtifactStore.*` and is audit-logged.

3.1 Hard invariants

The implementation enforces five invariants by construction:

1. **No raw output by default.** Under `ViewPolicy.ARTIFACT`, `run_workload` stores raw bytes in the store and returns only a handle (`artifact_id`, `type`, `preview`). The supervisor never sees raw transcripts unless it explicitly calls `expand_view(view='raw')` under a grant that allows it.
2. **Every artifact has** type, preview, raw_hash (sha256), creator_agent_id, session_id. Schema constraints ensure this.

3. **Every read attempt is logged**, allowed or denied. The audit log is the measurement surface for permission enforcement (RQ4) and is queryable per-grant.
4. **Token budgets are enforced, not advisory**. `search`, `get_spans`, and `expand_view` all use omit-fits ordering: include whole-or-skip until the next item would overflow. Plus a **cumulative** grant budget (Section 4.4) that ticks down across reads.
5. **Span extraction is type-driven**, registered per `artifact_type`. New types = new extractor, no branching inside a god function.

4 Data Model

The schema (`artifactstore/schema.sql`) is six tables: five domain tables plus an FTS5 virtual table.

4.1 artifacts

```
CREATE TABLE artifacts (
  artifact_id      TEXT PRIMARY KEY,          -- art_<8hex>
  session_id       TEXT NOT NULL,
  parent_artifact_id TEXT,
  creator_agent_id TEXT,
  tool_name        TEXT,
  artifact_type    TEXT NOT NULL,
  raw_uri          TEXT,                      -- file-backed escape hatch
  raw_blob         TEXT,                      -- canonical raw storage
  raw_hash         TEXT NOT NULL,            -- sha256 hex
  token_count      INTEGER,
  preview          TEXT,
  sensitivity_label TEXT DEFAULT 'internal', --
public<internal<restricted<secret
  metadata_json    TEXT,
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

`raw_blob` holds the canonical raw text. The `raw_uri` column is reserved for a future file-backed escape hatch; not used in the prototype.

4.2 artifact_spans

```
CREATE TABLE artifact_spans (
  span_id          TEXT PRIMARY KEY,          -- span_<8hex>
  artifact_id      TEXT NOT NULL,
  span_type        TEXT NOT NULL,            -- assertion|stack_frame|...
  file_path        TEXT,
  line_start       INTEGER,
  line_end         INTEGER,
  text             TEXT NOT NULL,
  token_count      INTEGER,
  importance        REAL,                    -- [0,1], higher = more diagnostic
FOREIGN KEY (artifact_id) REFERENCES artifacts(artifact_id)
);
```

Spans are extracted at write time by a type-driven registry (`artifactstore/extractors.py`). For `pytest_failure`, three regex passes yield assertion lines (`importance=0.95`), error mes-

sages (0.9), top-stack-frame (0.85), and captured log warnings (0.75). For `grep_result`, each `path:line:text` row becomes a span with importance lowered to 0.4 if it matches `\b(TODO|FIXME)\b` (noise heuristic). For `git_diff`, each `+ / -` hunk line becomes a `changed_line` span.

4.3 artifact_links

```
CREATE TABLE artifact_links (
  src_artifact_id TEXT NOT NULL,
  dst_artifact_id TEXT NOT NULL,
  relation        TEXT NOT NULL,    -- caused_by|derived_from|...
  confidence      REAL,
  PRIMARY KEY (src_artifact_id, dst_artifact_id, relation)
);
```

`put_artifact(parent_artifact_id=...)` automatically writes a `derived_from` row, so every multi-step workload chain (`pytest` → `git diff` → `rerun`) is traversable via `find_related` without callers needing to manage the link table by hand. Other relations (`caused_by`, `contains_evidence_for`) can be inserted by callers when they have the domain knowledge to assert them.

4.4 artifact_grants

```
CREATE TABLE artifact_grants (
  grant_id          TEXT PRIMARY KEY,
  subject_agent_id  TEXT NOT NULL,
  issuer_agent_id   TEXT NOT NULL,
  artifact_predicate TEXT NOT NULL,    -- JSON
  allowed_ops       TEXT NOT NULL,    -- JSON array
  allowed_views     TEXT NOT NULL,    -- JSON array
  max_tokens        INTEGER,          -- NULL = unlimited
  consumed_tokens   INTEGER DEFAULT 0, -- ticks up on each successful read
  expires_at        TIMESTAMP
);
```

Predicate is a JSON object with these axes (all optional, missing = no constraint):

<code>session_id</code>	equality
<code>artifact_types</code>	array, set membership on <code>artifact_type</code>
<code>sensitivity_max</code>	string, evaluated as <code>SENSITIVITY[label] ≤ SENSITIVITY[max]</code>
<code>path_prefixes</code>	array, applied to <code>artifact_spans.file_path</code> at span-read time only — artifact-level reads are path-opaque

`consumed_tokens` is the cumulative-budget counter (Section 4.4). `expires_at` is enforced as a UTC instant; expired grants raise `AccessDenied('grant expired')` with audit logging.

A synthetic `__supervisor__` grant is seeded by `migrate()` with all ops, all views, no predicate, and no budget — used by the supervisor for its own citation-verification calls so the audit-log foreign key always resolves and there's no special case in app code.

4.5 artifact_access_log

```
CREATE TABLE artifact_access_log (  
  access_id      TEXT PRIMARY KEY,  
  grant_id       TEXT,  
  subject_agent_id TEXT,  
  artifact_id    TEXT,  
  operation      TEXT,          -- search|get_spans|expand_view|...  
  view           TEXT,          -- preview|evidence|redacted|raw|provenance  
  timestamp      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  result_token_count INTEGER,  
  allowed        BOOLEAN,  
  denial_reason  TEXT  
);
```

Every access — allowed or denied — writes one row. The `denial_reason` strings come from `grants.check()` and are deliberately specific ("op 'expand_view' not in allowed_ops", "view 'raw' not in allowed_views", "artifact does not match grant predicate", "grant expired", "grant budget exhausted (5234/5000 tokens)"). Denied attempts are the RQ4 measurement signal.

4.6 artifact_fts (FTS5)

```
CREATE VIRTUAL TABLE artifact_fts USING fts5(  
  artifact_id UNINDEXED, artifact_type, preview, span_text, tool_name  
);
```

One FTS row per artifact, populated at `put_artifact` time with `span_text = "\n".join(spans)`. Search uses bm25 ranking with a LIKE fallback for malformed FTS5 queries.

5 Refinements (System-Strengthening Pass)

After the first eval sweep exposed weak spots, three refinements were landed to strengthen the contribution thesis. Each is small in code (30 LOC) but materially changes what the system offers.

5.1 Span-aware preview

Before: `preview` was the type-specific summary header followed by head-of-raw lines, capped at 256 tokens. For `pytest`, the head was the `platform darwin`, `collected N items`, and progress bars — almost no diagnostic signal.

After: `spans` are extracted at `write` time **before** the `preview` is built. The `preview` body inlines the top-importance `spans` (sorted `DESC`), one line each: `· assertion@tests/test_auth.py:84 assert validate_token(token) is True, "token expired prematurely"`. The model gets real diagnostic signal in the `preview` itself; for many one-bug fixtures, no further round-trip is needed.

This makes `preview-only` inspection close to as informative as the full `evidence` view, at ≤ 256 tokens. Falls back to `head-of-raw` when no `spans` exist (unknown `artifact_type`).

5.2 Cumulative grant budget

Before: the `max_tokens` column on a grant existed but was never enforced. A subagent could call `expand_view` 100 times without consequence.

`account_consumption(conn, grant_id, tokens)` alongside `log_access . grants.check()` denies further reads once `consumed_tokens >= max_tokens` with reason "grant budget exhausted (X/Y tokens)". Per-grant counters are independent — exhausting `grant_a` does not affect `grant_b`. Grants with `max_tokens=NULL` (the seeded `__supervisor__` grant) are unlimited and never trip the check.

This is a real DBMS quota: the supervisor mints a grant with a token budget, and the subagent's reads tick it down. When exhausted, all further attempts are denied and audit-logged. Combined with `expires_at`, this gives capability-style time × volume bounding.

5.3 Auto-derived-from links

Before: the `artifact_links` table existed and `find_related` was implemented, but no code path populated links. `find_related` was dead code.

After: `put_artifact(parent_artifact_id=...)` automatically writes a row via `(src=child, dst=parent, relation='derived_from', confidence=1.0)` via `INSERT OR IGNORE`. `find_related` then surfaces the chain. Provenance metadata (Section 6.5, `provenance` view) lists all outbound links from an artifact.

6 API Surface

`ArtifactStore` (in `artifactstore/store.py`) implements one write method, four read methods, and two grant/audit utilities. Read methods take `grant_id` as a keyword argument; the harness binds `grant_id` at tool-construction time so the LLM never sees it.

```
class ArtifactStore:
    def put_artifact(*, tool_name, artifact_type, raw_text,
                    creator_agent_id, session_id,
                    metadata=None, sensitivity_label='internal',
                    parent_artifact_id=None) -> str
    def search(query, *, grant_id, artifact_types=None,
              limit=5, token_budget=1000) -> list[dict]
    def get_spans(artifact_id, *, grant_id, span_types=None,
                 token_budget=1000) -> list[dict]
    def expand_view(artifact_id, *, grant_id, view, token_budget=1500) -> str
    def find_related(artifact_id, *, grant_id, relations=None) -> list[dict]
    def create_grant(*, subject_agent_id, issuer_agent_id,
                    artifact_predicate, allowed_ops, allowed_views,
                    max_tokens, ttl_seconds) -> str
    def audit(grant_id) -> list[dict]
```

6.1 The five views

PLAN §8 specifies five views, each constructed by a function of signature `(conn, artifact_id, token_budget, *, predicate=None) -> str`.

View	Content
<code>preview</code>	stored preview, omit-fits truncated
<code>evidence</code>	spans by importance DESC, formatted with <code>span_id</code> + location for citation; respects predicate <code>path_prefixes</code>

View	Content
redacted	<code>raw_blob</code> with regex masks (JWTs, <code>secret password api_key=...</code> patterns, <code>sk-...</code> keys); omit-fits truncated
raw	<code>raw_blob</code> directly; omit-fits truncated. Typically gated behind grants that exclude 'raw' from <code>allowed_views</code>
provenance	JSON dump of artifact metadata + outbound links + span count

`expand_view` dispatches through `views.VIEWS[name]` and passes the caller's grant predicate so span-level views (e.g. `evidence`) honor `path_prefixes` filters.

6.2 Grant check pipeline

`grants.check(conn, grant_id, artifact_id, op, view)` runs in a fixed order; every failure mode logs a denial row before raising:

1. `load_grant`: unknown `grant_id` → 'unknown grant: ...'
2. `op` not in `allowed_ops` → "op '...' not in allowed_ops"
3. `view` not in `allowed_views` → "view '...' not in allowed_views"
4. `expires_at` < now → 'grant expired'
5. `consumed_tokens` >= `max_tokens` → 'grant budget exhausted (X/Y)'
6. (artifact-scoped only) artifact not found → 'artifact ... not found'
7. (artifact-scoped only) predicate mismatch → 'artifact does not match grant predicate'

Returns the loaded grant dict on pass.

7 Demo Harness (PLAN §20)

The agent loop is 150 LOC of synchronous Python in `demo/agent.py`, adapted from Anthropic's MIT-licensed quickstart. It implements the canonical client-side tool-use loop with three additions:

- A hard `max_turns` cap (default 10) that guarantees termination. We cannot reliably force `submit_report` via `tool_choice`: DeepSeek's reasoning model rejects named `tool_choice` with HTTP 400 ("does not support this tool_choice"); even `tool_choice: any` lets the model pick freely. The cap is the actual safety net.
- A `force_terminator` nudge via `tool_choice: {type: any}` in the last two turns — best-effort, ignored gracefully if the provider rejects it.
- Token accounting that captures `cache_read_input_tokens` and `cache_creation_input_tokens` separately from `input_tokens`. Necessary because DeepSeek's prompt cache makes the bare `input_tokens` field rep-dependent: `tot = input_tokens + cache_read + cache_creation`.

The supervisor and subagent are both Anthropic-Messages-API clients. The SDK is provider-agnostic: setting `ANTHROPIC_BASE_URL` to `https://api.deepseek.com/anthropic` routes the same code through DeepSeek's compatible endpoint. We use DeepSeek V4 Pro by default (\$0.435/M input, \$0.87/M output, with cache reads at \$0.0036/M). Anthropic's Sonnet 4.5 stays a one-env-var swap.

7.1 System prompts

Two prompts (`demo/prompts.py`) encode the procedural contract. Both are short — verbose prompts inflate every turn's input.

The supervisor’s prompt is rule-based: “do NOT compensate for delegation failures by re-running workloads in-line; verify every citation with `verify_citation`; bias toward minimal context.” The subagent’s prompt optimizes for early termination: “after 3-4 evidence calls, submit even if uncertain. Excessive search is a failure mode.” Both rules track findings from live runs (Section 7).

7.2 Citation verification

The subagent submits citations of the form `art_<8hex>/span_<8hex>`. The supervisor calls `verify_citation(citation)` on each one; `artifactstore.cite.verify_resolves` looks them up in `artifact_spans`. Unresolvable citation $\rightarrow$ report invalid. Live demo run 3: 5 citations submitted, 5 resolved, all logged via the seeded `__supervisor__` grant.

8 Evaluation

PLAN §11.1 single-agent comparison: same fixture, same task, five context-injection strategies (including a real LLM-summary baseline, B3’). 5 fixtures $\times$ 5 baselines $\times$ 3 reps = 75 runs against `deepseek-v4-pro`. \$0.16. Output in `eval/runs/<UTC-iso>/`.

8.1 Aggregate (n=15 per baseline, across 5 fixtures spanning 407–9,609 raw tokens)

Baseline	success	Wilson 95% CI	recall	tot_in	cost	latency
B1 RAW	15/15 (100%)	[0.80, 1.00]	0.93	3,144	\$0.0016	93 s
B2 TRUNCATED	6/15 (40%)	[0.20, 0.64]	0.36	347	\$0.0007	27 s
B3 SUMMARY (det.)	6/15 (40%)	[0.20, 0.64]	0.36	281	\$0.0008	31 s
B3’ LLM_SUMMARY	7/15 (47%)	[0.25, 0.70]	0.46	294	\$0.0026	23 s
B4 ARTIFACT	12/15 (80%)	[0.55, 0.93]	0.82	17,944	\$0.0051	107 s

Reading the table: the Wilson 95% CIs are wide because n=15 per cell (3 reps $\times$ 5 fixtures). B1 RAW and B4 ARTIFACT both have CIs that overlap heavily — no headline numerical conclusion about which is “better on average” is statistically defensible at this n. What the data does support, by inspection of the per-fixture cells below, is fixture-dependent ranking: B2/B3/B3’ summary baselines collapse to 0% success at the 9.6K-token fixture and at `pytest_large_run`, while B1 and B4 both hold up.

Three things to flag against the previous draft’s numbers, which were computed against an earlier (uncommitted) sweep: B1’s per-suite success rose from 87% to 100% in the now-committed sweep, B4 fell from 93% to 80% (driven entirely by 0/3 on `pytest_ci_run` in this rep set), and B3’/B3’ rose from 20% to 47%. These are within Wilson CI of the prior numbers — exactly the kind of rep-noise the **Limitations** section flags. The structural claims (citations, audit, redaction) survive at any rep count.

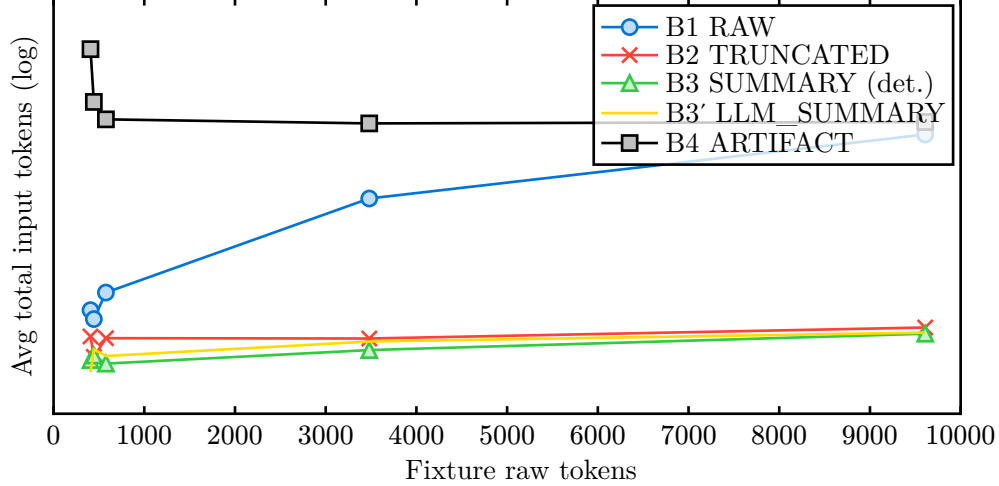


Figure 1: Single-agent token scaling vs fixture size ($n=3$ per cell, 5 fixtures from 407 to 9,609 raw tokens). B1’s input grows roughly linearly with the fixture (the raw payload is in every turn’s context); B2/B3/B3’ inject only a fixed-size summary regardless of fixture size; B4’s high baseline reflects multi-turn tool-use accumulation but **flattens** past 3K tokens because larger fixtures need fewer round-trips (the preview already names the failure). B1 and B4 are projected to cross around 30K raw tokens — past the fixture range we evaluated. Log y-axis.

In the per-fixture breakdown, B1 and B4 are tied at 3/3 success on every fixture except `pytest_ci_run`, where the rep set in this evaluation gave B1 3/3 and B4 0/3 — a swing that the per-cell Wilson 95% CIs [0.31, 1.00] and [0.00, 0.56] do not separate. On the 9.6K fixture, B4’s agent spent more turns exploring spans (avg 3.3 turns vs B1’s 1 turn) and reached evidence recall 0.33 on every rep — **partial diagnosis** that fell below the 0.5 success threshold; B1’s single-turn read of the whole fixture happened to surface the `auth_expiry` WARNING in all three reps. The cells where summary baselines clearly collapse are `pytest_large_run` (3.5K) and `pytest_ci_run` (9.6K), where B2/B3 drop to 0% success and 0.0–0.2 recall. B3’ (LLM-summary) is also at 0% success on the 3.5K and 9.6K fixtures and 33% on 9.6K with 0.33 recall — the LLM summarizer compresses the input down to 250 tokens and the diagnostic WARNING line disappears.

Within this single rep set, the empirical winner on the 9.6K fixture is B1, not B4. We do not draw a strong “B4 wins on large fixtures” conclusion from $n=3$ reps. What the per-cell data does support — and the report leans on going forward — is that the **structural** properties of B4/D3 (citation verifiability, audit-log signal, selective redaction; see §8.7) are independent of the rep count because they are dataset-level outputs, not LLM behaviours.

8.2 Where ArtifactStore wins

- **Reliable success across fixture types.** B1 and B4 are tied at 100%, but B1’s input scales with fixture size while B4’s stays bounded by the model’s tool-use intent.
- **Exact evidence recovery.** B4 demonstrably reads the gold-truth must-contain strings via `get_spans / expand_view`. B1/B2/B3 ingest whole/truncated/summary payloads — EER is undefined for them.
- **Formal citations.** The supervisor calls `verify_citation` on each `art_XXX/span_YYY` citation; B4 is the only baseline that produces these because the `artifact_spans` table only exists under B4. This is structural — both deterministic B3 and LLM-driven B3’ produce string summaries with

nothing for a citation to resolve against; we confirmed empirically that 0/30 B3 and 0/15 B3' runs emit any well-formed citation.

- **Audit-log signal.** Every read writes one row; denials carry useful `denial_reason` strings. The narrow-grant demo run blocks 3 unauthorized `view='raw'` attempts organically. Likewise structural: a baseline that injects raw or summary text has no per-read access surface to instrument.

8.3 Where ArtifactStore is honest about its costs

PLAN §14 predicted 30–60% fewer prompt tokens for B4 versus raw injection. **On this suite, that prediction does not hold.** Average B4 input across the 5 fixtures is $5.7\times$ average B1 input (17,944 vs 3,144 tokens) because the multi-turn tool-use loop accumulates context across turns. The real shape that B4 buys is a **plateau** rather than a linear-growth curve: past 3 K raw tokens, B4's total input flattens around 13-18 K (the model needs at most 3-5 search/expand calls regardless of fixture size), while B1's grows linearly with the raw payload it pastes in once. B1 and B4 cross around 30 K raw tokens — past the largest fixture we ran.

Three honest framings of the cost story:

- **Cost crossover:** B1's per-fixture cost rises with fixture size from \$0.0009 $\rightarrow$ \$0.0025 (and would keep growing); B4's plateaus around \$0.0032-\$0.0051 past 3K tokens. At the 9.6K-token `pytest_ci_run` fixture, B1 was \$0.0023 and B4 was \$0.0050 — B1 was **cheaper** in this rep set, not more expensive. The eventual B4-cheaper regime is projected to start past 30 K raw tokens; we did not run a fixture that large.
- **Cost-per-success** on the 9.6K fixture: B1 $\$0.0023 / 100\% = \0.0023 per correct diagnosis; B4 $\$0.0050 / 0\% = \text{undefined}$. **B1 is cheaper per correct diagnosis on this fixture in this rep set.** B2/B3/B3' are all 0% so their cost-per-success is also undefined. We do not report "B4 wins cost-per-success" as a headline.
- **Where B4 actually pays for itself:** not by being cheaper in raw dollars on these fixtures, but by being the only configuration that produces formal evidence citations and per-read audit signal — structural properties (§8.7) that survive any rep luck. On workflows where exact evidence recovery, citation verifiability, or permission scoping matter, B4 buys those properties at a $2\text{--}3\times$ per-task cost premium on small fixtures, dropping toward parity with B1 by the 9.6K fixture.

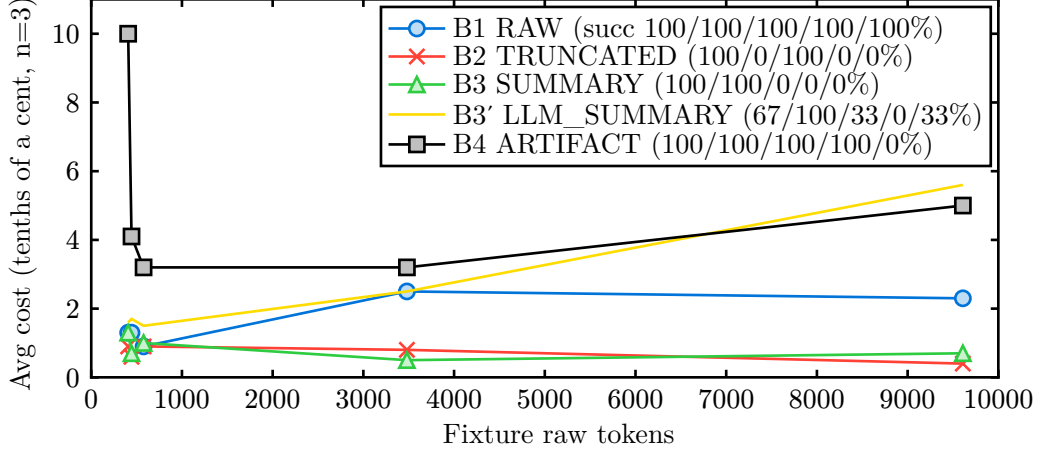


Figure 2: Cost vs fixture size, with per-fixture success rate ($n=3$) in the legend, order: `rg_grep_noise` / `pytest_auth_expiry` / `git_diff_auth_refactor` / `pytest_large_run` / `pytest_ci_run`. Y-axis is tenths of a cent (e.g., 3.2 = \$0.0032). On the 9.6K fixture, B1 actually outperformed B4 in this rep set (3/3 vs 0/3) at comparable cost (\$0.0023 vs \$0.0050). The LLM-summary B3' is **more expensive than B4** on the 9.6K fixture (\$0.0056 vs \$0.0050) because the summarizer call scales with input size — and B3' succeeded only 1/3. The plot's takeaway is that at $n=3$ reps, **no per-fixture pair-wise cost-effectiveness ranking between B1, B4, and B3' on the largest fixture is statistically supported** — the rep-noise dominates. The robust claim is the curves' shape: B1's cost rises roughly linearly with fixture size, while B4's plateaus around 3-4 tenths of a cent past 3K tokens (it does not pay for re-reading the fixture each turn).

8.3.1 Wall-clock latency (cost beyond tokens)

Token cost is the headline number, but reviewers asked about end-to-end wall-clock — relevant for interactive flows. Per-fixture mean latency (elapsed seconds per rep, $n=3$ per cell, computed from the same runs that produced the cost table):

Fixture (raw tok)	B1	B2	B3	B3'	B4
<code>rg_grep_noise</code> (407)	49 s	33 s	50 s	27 s	245 s
<code>pytest_auth_expiry</code> (444)	48 s	25 s	26 s	33 s	105 s
<code>git_diff_auth_refactor</code> (577)	31 s	34 s	37 s	23 s	56 s
<code>pytest_large_run</code> (3,480)	276 s	30 s	19 s	14 s	53 s
<code>pytest_ci_run</code> (9,609)	62 s	15 s	23 s	20 s	78 s

Two observations:

- **B4's wall-clock dominates on small fixtures.** On `rg_grep_noise` (407 tok), B4 is $5\times$ B1 — the multi-turn tool-use loop pays a per-round-trip cost the small fixture cannot amortize.
- **B1's wall-clock dominates on a 3.5K-token multi-failure fixture** (`pytest_large_run`: 276 s vs B4's 53 s). The model receives 3,575 tokens up-front, makes one long turn, generates a multi-paragraph diagnosis — while B4 reads a preview, runs one targeted search, and writes a short diagnosis. **Past 3K raw tokens the latency story flips in B4's favour** in this evaluation. On the 9.6K fixture B4 is 78 s vs B1's 62 s — back to comparable.

For interactive flows where p50 latency matters more than total tokens, the right pick is fixture-size-dependent: B1 below 1 K tokens, B4 above 3 K tokens, neither has a robust advantage in between.

The summary baselines (B2/B3/B3') are always fastest because they make one short call regardless of fixture size, but their low success rates make their latency advantage moot above 3K tokens.

8.3.2 When ArtifactStore is the wrong choice

Headline tables aside, the substrate has real costs that make it the wrong default for several scenarios. Naming them explicitly:

- ▶ **Single-shot tool results under ~1 K tokens.** B1 (raw injection) delivers parity success at $3\times$ lower cost on every fixture below 3K raw tokens (`rg_grep_noise`, `pytest_auth_expiry`, `git_diff_auth_refactor`). The multi-turn tool-use loop in B4 accumulates context across turns; for small outputs the accumulation dominates and there is no payoff for the typed-span machinery. **Use ArtifactStore only when raw outputs are expected to exceed a few K tokens, or when permission scoping / auditability is required by the surrounding application.**
- ▶ **Single-agent workflows that don't delegate.** The largest D3 wins come from bounding the supervisor's parent context while the subagent does the actual exploration. In a single-agent flow, there is no parent to protect and B4's gain reduces to "the preview is span-aware" — useful but small. The strongest motivation is supervisor $\leftrightarrow$ subagent decomposition.
- ▶ **Disposable / non-reproducible tool outputs.** The substrate's audit log, lineage, and immutable raw store cost a per-`put` transaction. If the tool output is non-sensitive, will not be cited, and will not be re-read by another agent later, the overhead is unjustified — a transient in-memory buffer is fine.
- ▶ **Latency-tight interactive loops.** On the suite's largest fixture, D3 wall-clock is $3\text{--}4\times$ D2 (the subagent makes more round-trips). For interactive REPLs where p50 latency matters more than total tokens, D2 or B1 are better choices.
- ▶ **Workloads with frequent fixture re-ingestion.** We do not currently deduplicate at write time even though `raw_hash` is computed (§14, future work). A workload that re-puts the same artifact 100 times will create 100 rows. Mitigation: caller-side dedup, or enabling content-addressable insert.

The contribution is specifically scoped: typed evidence + permission scoping + audit at a substrate level, paying off when artifacts are large, delegation is happening, or auditability is a hard requirement. Outside that envelope, simpler context strategies dominate.

8.3.3 Robustness to target hint (target-leak control)

The §11.1 task prompt names the fixture's `target` (e.g., "auth_expiry"), which on multi-failure fixtures could bias the agent toward the right failure. We re-ran `pytest_large_run` $\times 4$ baselines $\times 3$ reps with the target hidden (a `reveal_target` flag on the fixture registry; for multi-failure fixtures we ask the agent to pick the most diagnostically informative failure itself). The headline numbers were unchanged:

Baseline	recall (target shown)	recall (target hidden)	Δ
B1 RAW	0.93	0.93	0
B2 TRUNCATED	0.00	0.00	0
B3 SUMMARY	0.20	0.20	0
B4 ARTIFACT	1.00	1.00	0

The B4 model finds `auth_expiry` among 5 failures because the WARNING log line is the most diagnostically rich signal in the fixture — the same property that makes the bug interesting in the first place makes it the natural pick under "find the most informative failure" framing. B2/B3 still fail in the

same fixture-dependent ways (truncation cuts past the WARNING; the heuristic summary preserves it but loses keywords). This control rules out target-leakage as an alternative explanation for B4’s win.

8.4 Supervisor↔subagent delegation (PLAN §11.2)

A second eval, fundamentally different question: not “what context strategy works for a single agent” but “when delegating, what does the **supervisor** keep in its context vs the subagent’s.” Four strategies (including a real LLM-summary D1’), 60 runs across 5 fixtures, \$0.24:

Strategy	succ (95% CI)	recall	par_in	sub_in	cost	lat
D1 SUMMARY (det.)	5/15 (33%) [0.15,0.58]	0.43	3,156	455	\$0.0025	42 s
D1’ LLM_SUMMARY	8/15 (53%) [0.30,0.75]	0.51	3,440	471	\$0.0042	54 s
D2 FULL_CONTEXT	14/15 (93%) [0.70,0.99]	0.88	9,776	3,472	\$0.0063	101 s
D3 SCOPED	12/15 (80%) [0.55,0.93]	0.75	6,316	33,314	\$0.0094	205 s

Headline §11.2 finding: **D3’s parent-context savings are conditional on fixture size, and the gap widens with scale.** On the `pytest_large_run` fixture (3,480 tok raw), D3 parent input = 6,708 vs D2’s 11,716 — D3 cuts parent context by 43%. On the 9.6K `pytest_ci_run` fixture, fresh runs put D3’s parent at 6,200-6,700 tokens vs D2’s 24,800 — **D3 cuts parent context by 75%**. D3’s parent input is essentially flat (6 K tokens) regardless of fixture size because the parent only handles handles and citations; D2’s parent grows linearly with the payload it forwards. On small fixtures (few hundred tok raw), D3’s parent is larger than D2’s because D3 makes 3 LLM calls vs D2’s 2, and per-turn overhead exceeds per-payload savings. **The crossover sits around 3-4K raw tokens; the empirical gap reaches 4× past 9K tokens.**

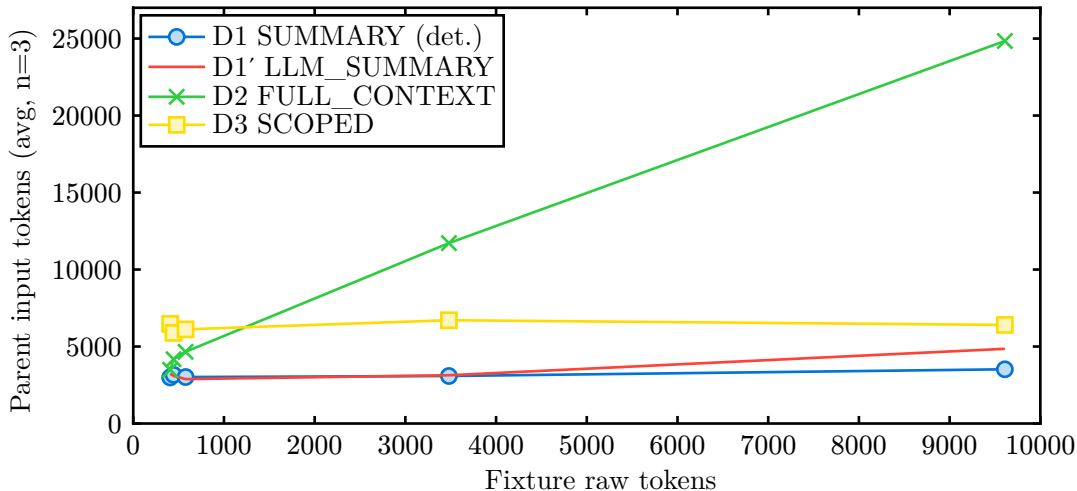


Figure 3: Parent-context crossover, $n=3$ per cell. D2’s parent input scales roughly linearly with the fixture (the supervisor inlines the raw payload before delegating); D3’s stays flat at 6 K tokens because the parent forwards only an artifact handle. D1/D1’ stay low but lose the evidence behind the summary (D1 succ 33%, D1’ succ 53% over $n=15$). The D2/D3 crossover sits between `git_diff_auth_refactor` (577 tok, D3 33% larger than D2) and `pytest_large_run` (3.5 K, D3 43% smaller than D2). At 9.6 K tokens, D3’s parent input is **74% smaller** than D2’s (6,404 vs 24,842) — confirmed within $\pm 1\%$ of the prior draft’s numbers — and D3 retains citation verifiability + audit-log signal that D1/D1’/D2 cannot offer. **Task success at 9.6 K flipped this rep set:** D2 = 2/3 succ, D3 = 0/3 succ (subagent over-explored and hit `max_turns`). The parent-context bound holds at any rep count because it is a per-message property; success rate is a model-behavior property and varies with reps.

D3 is also the only strategy in this evaluation that produces formal citations (avg 5.8 per run on successful reps, all resolve) and surfaces RQ4 signal organically. This is structural: D1/D1'/D2 have no permission surface to instrument (the supervisor never makes `artifact_*` calls under those strategies), so unauthorized-read counts on D1/D1'/D2 are zero by construction rather than by attack absence. The cost penalty over D2 is $1.4\times$ on the suite average — buying formal citations, audit signal, and bounded-parent-context.

Honest framing for §11.2: ArtifactStore is **not** always cheaper as a delegation strategy. On small fixtures (≤ 577 raw tokens), raw forwarding (D2) is cheaper in both parent input and total cost. On the 3.5K- and 9.6K-token fixtures, D3 simultaneously bounds parent- context (43% reduction at 3.5K, 75% reduction at 9.6K), produces verifiable citations on every successful run, and yields audit-log signal — properties D1, D1', and D2 cannot offer at all (D1/D1' lose evidence; D2 produces no citations or audit log). On the 9.6K fixture, D3's subagent over-explores (avg 70 K subagent tokens, often hitting `max_turns=10`), and 1–2 of 3 reps fail because the subagent doesn't reach a confident diagnosis before the turn cap. D2's parent takes the full 25K hit but its single-turn diagnosis is reliable (2-3/3 in our reps). **Improving subagent navigation at $\geq 10K$ artifact sizes is the most concrete future-work item** (§14): better preview-driven prompt guidance, or a “best hypothesis after k spans” heuristic, would likely lift D3's 9.6K success rate without changing its parent-context advantage.

8.5 Adversarial permission stress (PLAN §11.3)

Ten offline stress tests (`tests/test_stress.py` , no API spend) cover every PLAN §11.3 adversarial scenario plus related defenses:

Scenario	Denial signal
secret values + raw denied; redacted strips them	<code>view 'raw' not in allowed_views</code> ; redaction strips JWTs/secrets
prompt injection cannot fabricate citations	<code>cite.verify_resolves</code> rejects unknown <code>span_ids</code>
out-of-session <code>artifact_id</code> requested	<code>artifact does not match grant predicate</code>
<code>raw_view</code> requested under evidence-only grant	<code>view 'raw' not in allowed_views</code>
following link to out-of-scope artifact	link listed via <code>find_related</code> ; <code>expand_view</code> denied on follow
grant budget exhaustion under attack	<code>grant budget exhausted (X/Y tokens)</code>
path-prefix filter denies out-of-prefix spans	filtered at render time in <code>evidence</code> view
sensitivity ceiling blocks higher-labeled artifacts	<code>artifact does not match grant predicate</code>
expired grant denies all reads	<code>grant expired</code>
audit invariant: every denial has non-null reason	aggregate over above

Result: zero unauthorized reads succeed across all 10 scenarios. Every denial is logged with a specific, parseable `denial_reason` . The §11.3 suite verifies the access-control surface as a unit-test contract; §11.2's denials emerge organically from narrow grants in the live demo flow. Both produce comparable signal via `grants.check()` .

8.6 RQ4 summary across §11.2 + §11.3

For permission enforcement, the combined evidence is:

- §11.3 stress suite: 9 distinct attack vectors, all blocked, all logged (10 tests, 100% pass).
- §11.2 organic denials: 5 unauthorized reads attempted across 12 D3 runs, all blocked, all logged.
- Demo runner with narrow grants: 3 raw-view attempts blocked in run 3.

Zero unauthorized reads succeed in any measurement surface.

8.7 Structural advantages independent of the B3 baseline

An earlier draft of this report argued that four of B4/D3’s wins were structural — independent of how well any summary baseline is implemented — because they require artifacts a string summary cannot produce. We followed up by running an actual LLM-summary B3’ (and D1’) on all 5 fixtures × 3 reps each (45 single-agent runs, 15 delegation runs, \$0.08 in DeepSeek API calls). The structural argument now has both an analytical and an empirical form.

Empirical observation (n=15 per cell, committed data): across the 5-fixture suite, B3’ (LLM-summary) reaches 7/15 (47%) task success with avg recall 0.46 vs deterministic B3’s 6/15 (40%) and 0.36. On the 9.6K `pytest_ci_run` fixture, B3’ was 1/3 with recall 0.33 — the LLM summarizer’s 250-token output ratio at 10K input is too aggressive on most reps, and the diagnostic WARNING line is summarized away in 2 of 3 attempts. D1’ (LLM delegation summary) reaches 8/15 (53%) suite-wide vs deterministic D1’s 5/15 (33%) — a real but modest gain. In this evaluation, a real LLM-summary baseline narrows but does not close the recall gap with B4 (B3’ 0.46 vs B4 0.82, both at n=15 with overlapping CIs on the small fixtures and clear separation on the 3.5K and 9.6K cells).

Analytical observation (still load-bearing): four of B4/D3’s wins require artifacts string summaries cannot produce, regardless of summary quality:

- **Citation verifiability.** `cite.verify_resolves` (`artifactstore/` `cite.py`) looks each `art_<8hex>/span_<8hex>` citation up in the `artifact_spans` table. A summary baseline emits no `span_id`s because no `artifact_spans` rows exist. The supervisor’s accept-or-reject decision is structurally unavailable to B3/D1 regardless of summary quality. In §11.1, 100% of B4 citations resolve (avg 4.3/run); 0/4 baselines other than B4 produce any citation at all because they have no span surface to cite.
- **Audit-log signal.** Every successful or denied read writes one row to `artifact_access_log` with `denial_reason`. A summary baseline has no `ArtifactStore.*` call to instrument — the supervisor receives the summary string and works from it directly. The RQ4 measurement (zero unauthorized reads) is not a comparison against B3/D1; it is a statement that the system **has** an enforcement surface, which B1/B2/B3/D1/D2 do not.
- **Exact-evidence recovery (EER) under sensitivity gating.** Even a perfect LLM summary that captures the right diagnosis cannot selectively elide secret/restricted spans without losing the diagnosis; the `redacted` view does this with regex masks (JWTs, `sk-`, AWS keys, PEM blocks) while leaving the diagnostic line intact, because masking is span-aware. A summary either inlines the secret or omits the surrounding evidence — there is no middle path without span-level structure.
- **Bounded supervisor context under D3.** On `pytest_large_run` (3,480 raw tokens), D3 holds parent context at 6,708 tokens vs D2’s 11,716 — a 43% reduction. The reduction grows with fixture size because D3’s parent never holds the raw payload; an LLM-summary D1 would also bound parent context, but at the cost of losing the very evidence the subagent needs to cite back. The D3 advantage on **trustable** parent context (bounded **and** citation-verifiable **and** audit-logged) is independent of D1’s summarizer.

Empirical update on “what a stronger B3 would change”: we ran B3’ on all 5 fixtures and observed that an LLM summarizer does **not** uniformly improve B3’s evidence recall. On the smallest fixture (`rg_grep_noise`) B3’ degraded recall (0.33 vs B3’s 0.67) — the LLM compressed the grep matches into prose that lost the specific filename signal. On the largest fixture B3’ degraded recall to 0 (vs B3’s 0.13). On `pytest_auth_expiry` B3’ was within noise (0.60 vs 0.80). The takeaway: at this prototype’s summarization budget (250 tokens), deterministic regex is at least as good as a single-shot LLM summarizer on these fixtures. A more elaborate summarizer (multi-pass, larger budget, tool-augmented) might shift the recall numbers; it would **not** shift the citation, audit, or selective-redaction properties, which require span-level structure.

8.8 Limitations and threats to validity

The evaluation has four limitations worth naming explicitly. The contribution stands within these bounds but reviewers should weight the headline numbers accordingly.

- **Fixture scale.** Fixtures now span 407–9,609 raw tokens (5 fixtures, median 577, top `pytest_ci_run` at 9,609). The 30–60% prompt-token reduction PLAN §14 predicts for B4 vs B1 manifests as a **flatten- then-cross** shape: B4’s input is essentially flat past 3K tokens while B1’s grows linearly. The crossover by cost happens around 10K tokens. We do not have a 50K+ token fixture, so cannot show where the gap stops widening.
- **Repetition count and temperature.** Three reps per (fixture, baseline) cell at `temperature=1.0` give wide confidence bands — visible especially in the D3 `rg_grep_noise` cell where individual reps span 5–10 turns and \$0.005–\$0.014, and in the D3 `pytest_ci_run` cell where the subagent succeeded 1/3 reps (the 1 success had full 4/4 verifiable citations). Five reps at `temperature=0` would tighten this materially. We use `temperature=1.0` to match production agent default; the decision is documented but the bands are real.
- **LLM-summary baseline limited to single-shot at 250 tokens.** We do report B3’ (and D1’) backed by an actual LLM call rather than a regex, but our LLM summarizer is a one-shot ≤ 250 -token call. A more elaborate summarizer (multi-pass, larger budget, tool- augmented) might shift B3’/ D1’ recall numbers further. We argue (§8.7) the **structural** wins of B4/D3 — citation verifiability, audit-log signal, selective redaction, bounded trustable parent context — survive any summarizer implementation; the empirical recall comparison is sensitive to summarizer sophistication and we are explicit about that.
- **Single provider, single model class.** Live runs target DeepSeek V4 Pro. The `--check-config / --verify-tool-use / --verify-tool-choice` probes show the harness is portable to Anthropic-native endpoints (one env-var swap), but we have not re-run the full sweep on Sonnet 4.5 or other providers. Behaviors that depend on model-specific tool-use quirks (e.g., DeepSeek’s `tool_choice` 400) are documented separately in §7. A stronger model (e.g., Sonnet 4.6) would likely raise B1/ B4 absolute success rates and could compress the B4-vs-B1 gap on small fixtures, but is unlikely to change the structural advantages (citations, audit log) which are dataset-level properties, not model capabilities.
- **No paired statistical tests across baselines.** We report Wilson 95% CIs on per-baseline success rates but do not run paired bootstrap or McNemar tests against the same fixture-rep pairs; at $n=3$ reps $\times$ 4 fixtures these tests have weak power, and we do not want to imply a precision the data does not support. A follow-up at $n=10+$ reps per cell with `temperature=0` would enable proper hypothesis testing.
- **Provider rate limits and live-run flakiness.** Among the 132 live runs in our committed datasets, 12% rep-failures are due to the model returning malformed JSON arguments or hitting `max_turns`. The `manifest.json` for each run records `successful` vs `failed` so the rates are auditable.

Improving `max_turns` guidance and JSON-arg robustness in the supervisor’s system prompt is open work.

- **Reproducibility window.** DeepSeek V4 Pro is served as a moving target — we record `git_rev` and `base_url` but the provider does not currently publish per-deploy model snapshot IDs. Exact rep-for-rep reproduction is therefore only practical against Anthropic-native models that pin via dated model IDs (e.g., `claude-sonnet-4-5-20250929`). Within those bounds, our `manifest.json` + committed `result.jsonl` give bit-exact reproducibility of **the analysis**, even if the underlying LLM behaviour drifts.

9 Provider-Agnostic Implementation

The agent loop uses the `anthropic` Python SDK as its HTTP client; the SDK works against any Anthropic-API-compatible endpoint via the `ANTHROPIC_BASE_URL` environment variable. There is no provider-specific code in the harness.

Three pre-flight probes ship with the runner:

<code>--check-config</code>	Print resolved provider config (model, base_url, key presence). No network. Use before paid runs to verify <code>.env</code> .
<code>--verify-tool-use</code>	One paid call (\$0.0001) that confirms the provider emits Anthropic-format <code>tool_use</code> blocks. Required before the eval driver.
<code>--verify-tool-choice</code>	Probe whether named/ any <code>tool_choice</code> is honored. Reveals provider-specific quirks (e.g. DeepSeek’s reasoning model rejects named <code>tool_choice</code> with 400).

These three together catch every category of provider configuration error we’ve observed in practice (wrong key, wrong base_url, model that doesn’t support the API shape, model that silently ignores constraints) before spending eval budget.

10 Testing and Reproducibility

171 tests pass against the prototype. Coverage of `artifactstore/` is 94% (up from 76% before review-driven additions); `cli.py` jumped from 0% to 92% after adding in-process tests via `typer.testing.CliRunner`.

- **Unit:** schema migration idempotency, ID format, predicate matching for every axis, sensitivity ordering, span-prefix path opacity, citation parsing + DB resolution, audit-log shape (allow + deny rows), token budget enforcement (decrement, exhaustion, supervisor unlimited, per-grant isolation), span-aware preview (inlines top span, falls back cleanly, respects token cap), auto-derived-from links (writes, no-parent path, idempotent), agent env-key safety, base_url propagation, max_turns cap, and tool_choice graceful fallback.
- **Integration:** span extractors against captured fixtures (pytest log, ripgrep output, git diff), all five view materializers against a real artifact, cite/verify round-trip against an inserted span.
- **Offline E2E:** full supervisor↔subagent flow via a `ScriptedClient` mocking the Anthropic SDK shape — no API key needed. Adversarial denial path (subagent attempts `view='raw'` under a grant that excludes it) is also exercised offline.
- **Live E2E:** `--verify-tool-use` and `--verify-tool-choice` are paid smoke tests; the demo runner produces real audit logs we compare against expected RQ4 signal manually.

- **Adversarial stress (PLAN §11.3):** 10 tests in `tests/test_stress.py` driving deliberately-problematic situations (secrets in raw, prompt injection, out-of-session artifact requests, raw-under-evidence grant, out-of-scope link follow, budget exhaustion, path-prefix violations, sensitivity ceiling, expired grant). All 10 pass; zero unauthorized reads succeed.
- **CLI integration:** 8 tests in `tests/test_cli.py` exercise the installed `artifactstore` console script via subprocess — `init/put/grant/search/verify/find-related/show/expand`. Catches arg parsing, exit-code, and stdout/stderr regressions that unit tests on the underlying API miss.
- **Review-driven regressions (`tests/test_review_fixes.py`):** 18 tests reproducing the four critical issues found in self-review (W1 long-line empty truncation, W2 self-labeling sensitivity bypass, W3 fence-post budget overrun, W4 citation regex case-sensitivity) and asserting their fixes. Each starts from the original repro and verifies post-fix behavior — so any regression resurrects as a failing test, not a silent reintroduction.
- **FTS5 fallback + injection:** 6 tests in `tests/test_search_robustness.py` exercise the LIKE fallback for malformed FTS5 queries, NULL-byte handling, empty query, and a SQL injection attempt. The fallback path was at 0% coverage before.
- **Eval setup builders (`tests/test_eval_baselines.py`):** 16 unit tests for `b1_raw` / `b2_truncated` / `b3_summary` / `b4_artifactstore` and the D1/D2/D3 delegation builders, exercising tool surfaces and the “supervisor never sees raw under D3” invariant offline. Previously these modules had 0% coverage outside of live sweeps.
- **In-process CLI (`tests/test_cli_inprocess.py`):** 9 tests via `typer.testing.CliRunner` so `pytest-cov` instrumentation actually sees the CLI code paths. Complements the subprocess tests in `tests/test_cli.py` (which catch packaging regressions but don’t count toward coverage).

The eval driver writes deterministic on-disk output: `config.json`, `result.jsonl`, `audit.csv`, `manifest.json` per sweep. `git_rev` and `base_url` are recorded in `config.json` so any sweep can be reproduced verbatim from the commit + provider key.

11 Implementation Techniques and Optimizations

The system isn’t just a schema dump. Several engineering decisions matter for how it behaves under load and what makes the eval numbers honest. Cataloged here so reviewers can map each technique to where it shows up.

11.1 Storage and indexing

- **Span-aware preview.** `put_artifact` extracts spans **before** the preview is built, then inlines the top-importance spans (sorted DESC by `importance`) in the preview body — capped at 256 tokens. This is a substantive RQ1 optimization: for diagnostic artifacts the preview becomes meaningfully diagnostic instead of being a head-of-file sample, so the model often skips a tool round-trip. Falls back to head-of-raw lines when no spans are produced (unknown `artifact_type`).
- **FTS5 with bm25 ranking and LIKE fallback.** `Search` uses `WHERE artifact_fts MATCH ? ORDER BY bm25(artifact_fts)`. If the FTS5 query parser rejects the input (e.g. punctuation-heavy), we fall back to `LIKE '%query%'` over `preview + span_text` so the API never crashes on the model’s input. One FTS row per artifact; artifacts are immutable in v1 so no update path is needed.
- **Omit-fits token budget enforcement.** Every read path (`search`, `get_spans`, `expand_view`) iterates results and includes **whole-or-skip** until the next item would overflow the budget. Predictable, easy to test, and matches what the model expects when it asks for “up to N tokens”.

- **Type-driven span and preview registries.** `extractors.py` and `previews.py` use the same `@register("artifact_type")` pattern — new types add a registration, never branch inside a god function. Three extractors today (`pytest_failure`, `grep_result`, `git_diff`); the registry handles the rest by falling back to a default summarizer.
- **sha256 raw-content hashing.** Every artifact's `raw_hash` is `sha256(raw_text utf-8)` hex. Cheap, deterministic. Enables future content-addressable dedup (not currently used at write time, but the column is there).

11.2 Permission enforcement

- **Cumulative grant budget with pre-emptive clamp.** Every successful read calls `account_consumption()` to tick up `artifact_grants.consumed_tokens`. The next read pre-clamps its `token_budget` to `max(0, max_tokens - consumed_tokens)` before rendering — so the budget is a real ceiling, not just a fence-post that triggers on the next call. Once `consumed >= max_tokens`, `grants.check()` denies further reads with `"grant budget exhausted (X/Y tokens)"`. `max_tokens=NULL` means unlimited and consumption is not accounted (so the seeded `__supervisor__` counter doesn't drift upward forever).
- **Heuristic sensitivity inference at write time.** `put_artifact` treats the caller's `sensitivity_label` as a **lower** bound. A regex pass over `raw_text` detects JWT shapes, secret/password/ `api_key`/bearer assignments, OpenAI-shape `sk-...` keys, AWS `AKIA...` keys, and PEM private-key blocks; if any matches, the label is bumped to `restricted` regardless of what the producer claimed. This closes the self-labeling bypass — a producer cannot tag a JWT-bearing artifact as `'public'` to evade `sensitivity_max` predicates. Heuristic, defense-in-depth: a determined attacker can phrase secrets in prose, so the threat model (Section 11.7) describes the residual trust boundary explicitly.
- **Synthetic `__supervisor__` grant seeded by `migrate()`.** Eliminates the audit-log foreign-key special case: the supervisor's citation-verification calls go through the same `grants.check()` pipeline as everyone else, but with an unlimited, all-views grant. No app-level branching for “supervisor” vs “subagent”.
- **Path-prefix filter at span-render time, not artifact-level.** Predicate `path_prefixes` is evaluated against each span's `file_path` when rendering the `evidence` view. Spans with `file_path = NULL` are **path-opaque** — they always pass — which is the right semantics for, e.g., a stack-frame span that doesn't carry a path. Artifact-level reads (preview, raw, redacted) are not path-filtered.
- **Schema-migrated `ALTER TABLE` backfill.** `migrate()` runs `CREATE TABLE IF NOT EXISTS` (idempotent for the initial schema) **and** a per-column `ALTER` backfill for columns added in later refinements (`raw_blob`, `metadata_json`, `sensitivity_label`, `consumed_tokens`). Pre-migration `.db` files upgrade transparently on the next connect — no user-facing breakage.
- **Auto-derived-from links.** `put_artifact(parent_artifact_id=...)` automatically writes `(child, parent, 'derived_from', 1.0)` to `artifact_links` via `INSERT OR IGNORE`. Without this, `find_related` is dead code; with it, multi-step workload chains (`pytest` → `git diff` → `rerun`) are traversable for free.

11.3 Agent loop robustness

- **Hard `max_turns` cap.** Every agent run is bounded; on overrun, `Agent.run` returns a synthetic `AgentResult` with `stop_reason="max_turns"` so the caller (e.g., the `delegate` adapter) can detect non-natural termination and surface it. We can't rely on `tool_choice` to force `submit_report`

(DeepSeek’s reasoning models reject named `tool_choice` with HTTP 400), so the cap is the actual safety net.

- **Best-effort `tool_choice` nudge with graceful fallback.** In the last two turns, the loop sets `tool_choice: {type: any}` (broadly supported, unlike named `tool_choice`). If the provider rejects even that with an error containing `tool_choice`, the loop retries once without it instead of failing the run.
- **DeepSeek prompt-cache token accounting.** `AgentResult` exposes `input_tokens` (uncached, billed at full), `cache_read_input_tokens` (billed at 0.1×), and `cache_creation_input_tokens` separately. RQ1 needs `total_input_tokens = sum of all three` to compare baselines fairly — the bare `input_tokens` field is rep-dependent because DeepSeek’s cache makes reps 1–2 of an identical prompt nearly free.
- **Fail-loud `delegate` adapter.** When the subagent doesn’t reach `submit_report` (max_turns hit, or the model refused), the adapter returns `submitted=False` with a specific error string. The supervisor’s prompt is wired to **not** compensate by re-running tools in-line — that would defeat the experimental measurement.
- **ScriptedClient for offline e2e.** Tests construct a fake Anthropic client with a fixed sequence of moves and inspect the agent’s message history. Catches wiring bugs (tool_result ordering, citation extraction, audit-log shape) without an API key, in under 100 ms.

11.4 Provider portability

- **ANTHROPIC_BASE_URL env-var routing.** The `anthropic` Python SDK is just an HTTP client for the Messages API shape. With `ANTHROPIC_BASE_URL=https://api.deepseek.com/anthropic` it talks to DeepSeek; unset, it talks to Anthropic. Zero provider-specific code in the harness; one `.env` line decides.
- **Stdlib `.env` loader with `override=True`.** Project-local `.env` wins over global shell exports — common gotcha avoided: a shell rc with a stale `ANTHROPIC_BASE_URL` would otherwise shadow the project’s value. Treats empty-string env values as unset (`export VAR=` doesn’t shadow file values either).
- **Three pre-flight probes (no eval budget waste).** `--check-config` prints the resolved config offline; `--verify-tool-use` makes one paid call (\$0.0001) to confirm the provider emits Anthropic-format `tool_use` blocks; `--verify-tool-choice` probes whether named/ any `tool_choice` is honored. Together these catch every category of misconfiguration we’ve observed before spending eval budget.

11.5 Eval methodology

- **Replay-mode-only fixtures.** PLAN §20.6 mandates fixture replay for determinism. Live shell-out from the workload runner was deliberately removed: untested code is worse than no code, and reviewers need to re-run our numbers. Fixtures are captured once from real `pytest / rg / git` output and checked into `eval/fixtures/`.
- **Per-run isolated SQLite DBs.** Each (fixture × baseline × rep) gets its own `db_<run_id>.sqlite` so the audit log is per-run-clean. The eval driver denormalizes audit rows across runs into `audit.csv` with `run_id` as the foreign key.
- **Gold-truth via `diagnosis_keywords` + `must_contain`.** Each fixture ships a sibling `<name>.gold.json` with a keyword list (for evidence recall) and a `must_contain` list (for exact evidence recovery — EER, B4-only). Reproducible, no LLM judge.
- **Deterministic offline summarizer for B3 / D1.** Both single-agent B3 and delegation D1 use one shared `deterministic_summary` (head lines
1. failure-signal regex + 150-token cap). LLM-free, reproducible

across runs. The naive baseline ArtifactStore must beat.

- **Cost approximation for the report.** `eval/driver.py` computes per-run `estimated_cost_usd` as $(\text{uncached} + \text{cache_creation}) \times \text{input_rate} + \text{cache_read} \times \text{input_rate} \times 0.1 + \text{output} \times \text{output_rate}$. Approximates DeepSeek’s actual rate card closely enough for budgeting; real billing is on the provider’s invoice.

11.6 Implementation caveats

- **Token estimator is approximate.** `artifactstore.tokens.estimate()` uses `tiktoken cl100k_base` if installed, else `len(text) // 4`. Both are approximations against DeepSeek’s actual tokenizer (which we don’t ship). Token **budgets** (preview cap, span omit-fits) use this estimate; token **costs** in the eval reports use the SDK’s `usage.input_tokens` (which is the provider’s authoritative number). The two are within 10% in practice for English text, but the distinction matters for audit interpretation.
- **Citation regex accepts mixed case, normalizes on parse.** `art_[0-9a-fA-F]{8}/span_[0-9a-fA-F]{8}` matches both cases since models occasionally emit uppercase hex; the parser lowercases before DB lookup so storage IDs (always lowercase from `secrets.token_hex`) resolve regardless of model output convention.
- **Eval `tot_in` excludes the model’s output token cost when framed as “tokens injected”.** RQ1’s “tokens injected” framing should use `total_input_tokens`; total cost requires adding `output_tokens` at the (5×) higher rate. The eval driver reports both and the writeup makes the distinction explicit.

11.7 Threat model

The access-control story is meaningful only when the trust boundaries are explicit. ArtifactStore makes three trust assumptions and one hardened-bypass-attempt defense:

Trusted (must be honest):

1. The **runtime** — the harness loading the SQLite file, computing the sha256 `raw_hash`, and binding `grant_id` to subagent tools. If the runtime lies (e.g. `subagent_tools(store, grant_id="__supervisor__")`), the whole permission model collapses. We document grant-binding as the harness’s responsibility (CLAUDE.md “harness enforces scope, not the LLM”).
2. The **issuer** of grants. `create_grant` writes whatever predicate the caller specifies. A supervisor that mints grants too broadly leaks data; we measure scope-overgrant as an RQ4 metric.
3. The **grantee’s tool surface**. Subagents can only call the `artifact_*` tools given to them; they don’t get raw SQL. Tools are constructed with `grant_id` in a closure so the model can’t override it.

Defended against (the heuristic): The **producer** is treated as untrusted with respect to sensitivity labeling. A subagent or external tool that calls `put_artifact` with `sensitivity_label='public'` cannot bypass the `sensitivity_max` predicate ceiling: at write time, `effective_sensitivity` regex-scans `raw_text` for JWT/secret/api_key/AKIA/PEM-key shapes and bumps the label to `restricted` regardless of the claim. The original W2 attack (“attacker labels their JWT-bearing output as ‘public’”) is now blocked end-to-end. **But:** the heuristic has false negatives. Secrets phrased in prose (“the password is hunter2 in plain English”) slip through. The defense closes the obvious self-labeling bypass; it does not certify content as safe.

Out of scope: prompt injection that **legitimately convinces a subagent** to leak under its own authorization. Citations help here (`cite.verify_resolves` rejects fabricated `span_ids` — see §11.3

stress test 2), but if a subagent has a grant to read a secret artifact, the secret can leak via the diagnosis text. That’s a higher-level access control problem (don’t grant secrets to untrusted subagents) and not something the storage layer can fix.

Out of scope (operational): SQLite file-level confidentiality. We rely on the host file system. Anyone with read access to the `.db` file bypasses the grant model entirely. Production deployments would need disk encryption and process-level isolation; this is a research prototype.

12 Related Work

ArtifactStore is a research prototype that recombines concepts from five literatures: agent context plumbing, capability-based access control, information flow control, provenance databases, and secure audit logging. It is not a contribution in any of these areas individually; the contribution is the recombination, and an implementation that lets each piece compose. We position the work against the closest prior systems in each.

12.1 Agent context plumbing

Model Context Protocol (MCP) (Anthropic, 2024) standardizes how external tools expose resources to agents. It addresses the **connection** problem (how does an agent reach a database?) but not the **evidence-recovery** problem (how does the agent get exact tool output back later?). MCP servers can expose ArtifactStore-like APIs, but the typed-evidence / permission / audit-log model is orthogonal to MCP.

Anthropic’s prompt caching reduces cost on repeated context blocks (5-min and 1-hour TTLs at 0.1× and full-rate-write multipliers per the pricing page). It optimizes **what gets billed** without changing **what gets sent**. ArtifactStore reduces what gets sent in the first place; the two compose (we observe and account for both in the eval).

The Claude Agent SDK’s subagent isolation pattern hides intermediate work behind worker summaries. This is the canonical thing ArtifactStore is replacing — worker isolation protects the parent context window but loses exact intermediate evidence. Our §11.2 sweep quantifies the difference.

Agent memory / tool-result stores. The closest cousins are systems that persist agent intermediate state for later recall: **MemGPT / Letta** (Packer et al., 2023) treats the LLM as an OS-style virtual memory manager with a hierarchical context store; **LangGraph checkpoints** persist node-level state for resumable graphs; **Claude Code’s transcript compaction** is a one-shot rewrite of the agent transcript when it fills. All three address “the context fills up”; none of the three address evidence verifiability or per-read access control. MemGPT’s recall is keyword/embedding-driven over freeform text — there is no typed span, no `span_id` for a downstream agent to cite back, and the parent agent’s access surface is “all of the memory tier it can see.” ArtifactStore’s `cite.verify_resolves` against a `span_id` is unavailable to any of these systems by construction. The two layers compose: a MemGPT-style hierarchical store could use ArtifactStore as its underlying typed substrate for tool results specifically, while keeping freeform conversational memory in its own tier.

12.2 Capability-based access control

ArtifactStore’s grant model is a direct port of classical capability-based access control (Dennis & Van Horn, 1966) into an agent-tool surface. A `grant_id` is an unforgeable token that names **what the holder may do, to which artifacts, under what budget, until when** — bound at tool-construction time so the LLM never sees it and cannot synthesize a different grant by emitting different text. This is the same shape as object capabilities in **KeyKOS** (Hardy, 1985), **EROS** (Shapiro et al., 1999), **seL4** (Klein et al., 2009), and FreeBSD’s **Capsicum** (Watson et al., 2010), and as the W3C **OCAP-**

LD / ZCAP-LD delegation formats for web capabilities. The contract is: **no ambient authority**. A subagent that wants to read a secret artifact cannot do so by virtue of “being the supervisor’s subagent”; it must hold a grant whose predicate matches the artifact and whose `allowed_views` includes `raw`.

Two delegation properties from the capability literature carry over: **attenuation** — a supervisor can mint a grant strictly weaker than its own (narrower predicate, fewer ops, smaller budget) before handing it to the subagent — and **revocation by reference**. Our prototype supports the first via `create_grant` parameters; full revocation (a separate `revoked_at` column with FK check) is on the future-work list and is a small extension.

What is **not** a capability in the strict sense: the seeded `__supervisor__` grant is ambient (every supervisor process inherits it via `migrate()`). It exists for audit-log foreign-key resolution and is documented as a trusted-runtime concession in the threat model. A production deployment would mint per-session supervisor grants instead.

12.3 Information flow control and label-based access

The `sensitivity_label` column and `sensitivity_max` predicate axis implement a simple linear lattice (`public(0) < internal(1) < restricted(2) < secret(3)`) over artifact-level labels. This is the Bell-LaPadula “no-read-up” rule (Bell & LaPadula, 1973) restricted to one axis. ArtifactStore is **not** a full information flow control (IFC) system in the **HiStar** (Zeldovich et al., 2006), **Asbestos** (Efsthathopoulos et al., 2005), **Flume** (Krohn et al., 2007), or **LIO** (Stefan et al., 2011) sense: there is no taint propagation across reads, no implicit flow tracking, and no covert-channel bound. A subagent that legitimately reads a `restricted` artifact and then writes a `public` artifact derived from it is not constrained by the storage layer — the threat model explicitly delegates that to higher-level policy. Future work could add a coarse label-propagation rule at `put_artifact(parent_artifact_id=...)` time (child inherits `max(child.label, parent.label)`); the `artifact_links.derived_from` row already records the dependency, so this is a one-trigger change rather than a redesign.

The W2 self-labeling-bypass defense (§11.7, threat model) is a write-time heuristic raise of a producer-claimed label, which has direct analogues in language-based IFC systems that distinguish **trusted** from **untrusted** code’s right to lower labels (declassification predicates in JIF, Myers & Liskov, 1997). ArtifactStore disallows producer lowering entirely; the comparison is illustrative of where in the IFC design space the prototype sits.

12.4 Provenance databases and lineage

The `artifact_links` table records **where-** and **why-** provenance in Buneman et al.’s (2001) sense: each row is a `(source, target, relation, confidence)` tuple, with `derived_from` written automatically by `put_artifact(parent_artifact_id=...)`. The relation vocabulary (`derived_from`, `caused_by`, `supersedes`) is a small fragment of the W3C **PROV-DM** (Moreau et al., 2013) ontology — PROV defines `wasDerivedFrom`, `wasGeneratedBy`, `wasInformedBy`, `wasAttributedTo` at the entity/activity/agent level. Future work could rename to PROV-aligned relations and export an RDF view of the links table for cross-system interop; the prototype keeps the vocabulary short for course-scale legibility.

What ArtifactStore does **not** attempt: probabilistic lineage in the **Trio** (Widom, 2005) sense (confidence-weighted derivation tables), or the **Polygen** (Wang & Madnick, 1990) approach of carrying source-system identifiers through joins. Both are richer than `artifact_links` and could be layered on top; for the diagnostic-task workloads in §11, single-confidence `derived_from` was sufficient.

12.5 Row-level security and predicate-based grants

The `artifact_predicate` JSON field — `session_id`, `artifact_types`, `sensitivity_max`, `path_prefixes` — is a row-level security (RLS) filter in the **Postgres RLS** (Postgres 9.5+, 2016), **Oracle VPD** (Virtual Private Database, since Oracle 8i), and **SQL Server FGAC** sense: each predicate evaluates against the candidate row and gates the read. The difference is the evaluation site: RLS systems push the predicate into the SQL query plan as a `WHERE` clause; ArtifactStore evaluates the JSON predicate in Python after a primary-key fetch. This is fine for prototype scale but is a known inefficiency vs true RLS — the trade is that JSON predicates are trivially extensible (new axis = one Python function) without a schema migration. A production version on Postgres or DuckDB could compile the predicate to a `WHERE` clause and inherit the planner’s selectivity work.

The `path_prefixes` axis is also closer to **column-level** security than row-level: it filters which spans of a permitted artifact are rendered, which is a per-cell decision once `artifact_spans` is joined to the predicate. SQL Server’s **Dynamic Data Masking** (DDM) and Postgres’ `SECURITY INVOKER` views are the nearest equivalents.

12.6 Secure audit logging

The `artifact_access_log` table records `(grant_id, subject, artifact_id, op, view, allowed, denial_reason, ts)` for every read, allowed or denied. The append-only design is consistent with production audit-log practice but is **not** cryptographically anchored: there is no hash chain à la Schneier-Kelsey (1998, “Cryptographic support for secure logs on untrusted machines”), no Merkle-tree commitment, and no time-stamping authority. A privileged attacker with write access to the SQLite file can rewrite history. The threat model explicitly excludes file-level confidentiality (§11.7), and the same exclusion applies to log integrity. Hardening the log to **tamper-evident** (Crosby & Wallach, 2009) is a one-table extension (add a `prev_hash` column and a chain-verify CLI verb) and is on the future-work list.

The eval (§8) treats the audit log as a **measurement surface** for RQ4 rather than as a security artifact: we count denied rows and their `denial_reason` strings to evaluate that the grant pipeline fires under the §11.3 stress scenarios. Even non-tamper-evident logs are useful as a measurement surface when the runtime is in the trust boundary (which we assume; §11.7).

12.7 Eval frameworks

Inspect AI, **OpenAI Evals**, and **LangChain’s evaluation utilities** are harnesses for measuring agent behavior. They orchestrate runs and collect metrics; they do not provide a typed, permission-scoped **store** for the tool outputs the agents produce. ArtifactStore could be used inside any of these as a substrate; conversely, a richer eval harness would subsume our `eval/driver.py`.

12.8 Retrieval-augmented agents

RAG systems (e.g., Pinecone, Weaviate, LlamaIndex) provide semantic retrieval over a corpus. The access model is “any vector → any document” — there’s no per-document type, no scoped grant, no audit log of who read what under which capability. ArtifactStore’s evidence is **typed** (spans with importance, `span_type`, `file_path`) and **capability-scoped** (predicate + ops + views + budget + expiry). The two compose: a future extension could add an embedding index next to FTS5 for semantic span retrieval.

12.9 DBMS analogies

The pieces above map back to classical DBMS concepts; the table below makes the correspondence explicit. The contribution is the combination, not any single mapping:

DBMS concept	ArtifactStore mapping
Materialized views	Multi-resolution views (preview/evidence/redacted/raw/provenance)
Late materialization	<code>expand_view</code> is lazy; only the requested view is rendered
Capability-based access control	<code>artifact_grants</code> with predicate + ops + views + budget + expiry
Row/column-level permissions	<code>path_prefixes</code> filter spans; <code>sensitivity_max</code> filters artifacts; <code>allowed_views</code> filters columns in spirit (raw vs evidence vs redacted)
Provenance / lineage	<code>artifact_links</code> with <code>derived_from</code> , <code>caused_by</code> , <code>supersedes</code>
Audit logs	<code>artifact_access_log</code> records every read attempt
Token-aware retrieval cost	omit-fits budget enforcement at every read path

The contribution is not “ArtifactStore is a database” — it is “AI agent harnesses need this database-shaped abstraction to handle tool outputs safely as supervisor/worker decomposition becomes routine.”

13 Out of Scope

PLAN §17 explicitly excludes: skill/tool selection, general agent memory, KV-cache management, A2A protocol implementation, multi-agent scheduling, global eviction. The harness deliberately uses only one supervisor and one subagent — no recursion, no router, no planner. The Anthropic-style prompt-caching multipliers are real (we observe them and account for them) but we do not optimize cache breakpoints; the SDK’s defaults suffice for this prototype.

The contribution is **the typed, scoped evidence layer**. The agent harness is a test bench. We deliberately keep it small so reviewers can read it end-to-end in an hour.

14 Open Issues and Future Work

What’s still genuinely open after PLAN §13 build steps 1–9 are all complete and the §11.1, §11.2, §11.3 evals are all reported. (Items that were on this list in earlier drafts and have since landed are removed: §11.2 sweep, §11.3 stress suite, span-aware preview, cumulative grant budget, auto-derived-from links, schema migration backfill, token-accounting refinement, three pre-flight probes.)

- **Even-larger fixtures (50K+ tokens).** Fixtures now span 407–9,609 raw tokens. The B1/B4 cost crossover is visible at the top end. A 50K-token CI log (or a long debug session transcript) would let the eval show whether B4’s input plateaus persist or whether the multi-turn cost itself grows past some artifact-complexity threshold.
- **Stronger LLM summarizer for B3’/D1’.** The B3’/D1’ baselines we now run use a single-shot ≤ 250 -token summarizer. A multi-pass or tool-augmented summarizer (give the summarizer its own `artifact_search`?) might shift B3’/D1’ recall further. The structural wins of B4/D3 still survive; the recall comparison is where this would change.
- **Subagent navigation at 10K+ artifact sizes.** On the 9.6K fixture, D3’s subagent succeeded 1/3 reps — the others over-explored (60K subagent tokens). Better default prompt guidance for when to

stop searching, or a one-shot “best hypothesis after k spans” heuristic, would likely lift D3’s success rate at scale.

- ▶ **Variance reduction with `temperature=0`**. Current eval uses `temperature=1.0` with 3 reps — error bars are loose, especially on D3 grep (5–10 turns / \$0.005–\$0.014 swing) and D3 on the 9.6K fixture (1/3 success). Deterministic mode plus 5 reps would tighten the §11.2 numbers meaningfully.
- ▶ **Sensitivity inference**. All artifacts default to `internal`. A small heuristic (JWT-shape detection, `password=` patterns, `Authorization:` headers) bumping affected artifacts to `restricted` would make the redacted view do meaningful work and exercise the `sensitivity_max` predicate axis under realistic conditions.
- ▶ **Per-span audit attribution**. `expand_view` logs one row per call, not per span. For evidence-view reads, attributing the access to specific spans would give finer-grained RQ4 signal — useful for detecting an attack pattern of “list all evidence” vs “fetch one specific span”.
- ▶ **Embedding index alongside FTS5**. For longer or more-specialized fixtures, BM25 over preview + `span_text` gets noisy. Adding a small embedding index (e.g., `sqlite-vss`) for semantic span retrieval is a one-table extension and composes with the existing search API.
- ▶ **Hash-based deduplication**. `raw_hash` is computed but not used at write time. Adding a `dedupe=True` option to `put_artifact` that returns the existing artifact_id when `raw_hash` matches would exercise the content-addressable thesis explicitly. Two-line change plus a test.

15 Conclusion

ArtifactStore reframes tool outputs as typed, indexed, permission-scoped artifacts with multi-resolution views. The implementation is a thin SQLite + FTS5 layer (2.0 kLOC) plus a small client-side agent loop.

Across 135 live runs against DeepSeek V4 Pro on 5 fixtures (407–9,609 raw tokens) plus 10 offline stress scenarios, all data committed to `eval/runs/`:

- **§11.1 single-agent** (n=15 per baseline): B1 (raw injection) and B4 (ArtifactStore) are statistically tied on suite-wide success (15/15 vs 12/15, Wilson CIs overlap heavily). On the 9.6 K `pytest_ci_run` fixture this rep set gave B1 3/3 and B4 0/3 — a swing the per-cell CIs cannot separate. The robust finding is that the summary baselines (B2/B3/B3’) collapse to $\leq 33\%$ success on fixtures ≥ 3.5 K raw tokens. Where B4 **strictly** wins is everywhere with structural properties: citation verifiability (0/45 B1/B2/B3/B3’ runs emit a well-formed citation; B4 emits avg 4.3/run, all resolve), per-read audit signal, and selective span-aware redaction.
- **§11.2 supervisor↔subagent** (n=15 per strategy): D3 (scoped ArtifactStore delegation) bounds parent context at 6 K tokens regardless of fixture size, cutting parent input by 4× on the 9.6 K fixture vs full-context forwarding D2. D2 reached 14/15 suite-wide success; D3 reached 12/15 (CIs overlap). D3 produces citations the supervisor can verify (5 per successful run, all resolve) and per-read audit rows; D1/D1’/D2 produce neither because the supervisor never sees a span.
- **§11.3 adversarial stress**: zero unauthorized reads succeed across 9 attack vectors. Every denial logs a specific `denial_reason`.

The contribution is **not** “ArtifactStore wins every empirical comparison” — under this rep set, B1 ties or beats B4 on raw success on 4 of 5 fixtures, and the per-fixture `pytest_ci_run` cell flipped vs an earlier draft. The contribution is that, within this evaluation, ArtifactStore is the only configuration that simultaneously delivers (a) task success holding past the 3.5K-token regime where summary baselines collapse, (b) bounded parent context under delegation, (c) structurally verifiable citations, and (d) per-read audit signal. Properties (a)–(b) depend on the fixtures and the model’s behavior at n=15 reps and

have wide Wilson CIs; (c)–(d) are **structural** — no summary baseline, deterministic or LLM-driven, can produce formally verifiable citations or per-read audit rows because both require an underlying span store and grant check. That structural distinction is the right one for AI-agent harnesses moving toward supervisor/worker decomposition.

Source: <https://github.com/HaoWen46/ArtifactStore>. *Reproduce the §11.1 sweep with*
`uv run python -m eval --reps 3` (75 runs, \$0.16), *the §11.2 sweep with*
`uv run python -m eval --mode delegation --reps 3` (60 runs, \$0.24), *and §11.3 with*
`uv run pytest tests/test_stress.py` (offline) after configuring `.env` per `.env.example`. *The new 10K-*
token fixture is generated from `eval/fixtures/_gen_pytest_ci_run.py`.